

Do English Hallucination Detectors Work in Other Languages?

LLM Judges Against HHEM and LettuceDetect on a Planted-Error Benchmark in Eleven Languages

Aghasalim Mustafazada¹ · Teymur Eyvazov²

¹ Co-CEO and Head of AI, ² Co-CEO and Head of Operations — SIBA, "Süni İntellekt Biznes Avtomatlaşdırma" MMC, Baku, Azerbaijan · September 2026 · v2

Abstract. Retrieval-augmented assistants fail their users most often by stating a price, hour, address or phone number that the documents they were given do not support. The open-weights detectors built for this failure — Vectara HHEM-2.1-Open and LettuceDetect — are trained on English. We ask whether they transfer, and whether the plain alternative, a general language model asked to list the unsupported claims in the answer's own language, does. We build a planted-error benchmark: two businesses (a dental clinic, an electronics shop), eleven languages across five scripts (English, Azerbaijani, Russian, Turkish, Ukrainian, Kazakh, Arabic, Persian, Hindi, Indonesian, Vietnamese), and for each a grounded answer plus seven single-error variants — a wrong price, wrong hours, a wrong street number, one phone digit changed, a wrong return window, an invented Sunday opening and an added claim — planted by slot substitution so every language receives identical errors: 154 answers, 132 of them wrong. Two open LLM judges (openai/gpt-oss-120b, gemini-3.8-flash) catch all 132 with no false alarms; a 27B open model catches 131 in a median 0.36 s. HHEM-2.1-Open reaches 79 % recall, but its consistency score falls below the threshold for any non-English text: in Russian, Arabic, Persian and Indonesian it flags every answer, grounded or not, so its recall measures the language, not the claim; in English it catches 5 of 12. LettuceDetect catches 49 % overall, 8 of 12 in the Latin-script languages and 2 of 12 in Kazakh and Hindi. We also document a measurement pitfall in which an empty reply from a thinking model scores as "grounded", and the package fix. Code, data, raw outputs and the runner are at github.com/aghasalim/groundedness (MIT; *pip install groundedness*).

Author contributions. A.M. designed the benchmark, wrote the package and the cases, ran the experiments and wrote the paper. T.E. contributed the operational setting, the business requirements and review.

1. Problem

A business assistant answering from its owner's documents has one failure that costs money: it says something the documents do not say. The tools built to catch it in retrieval-augmented generation are trained and evaluated on English — HHEM-2.1-Open, a cross-encoder trained on factual-consistency data; LettuceDetect, a ModernBERT token classifier trained on RAGTruth; Patronus Lynx, a Llama-3 fine-tune under a non-commercial licence. Their model cards list one language. A developer serving Azerbaijani, Kazakh or Vietnamese users must either assume they transfer or use nothing.

This report measures whether they transfer, on the same cases as the alternative that needs no training data: asking a capable general model, in the answer's language, which claims the sources do not support.

2. Method

Cases. For each language a native-form facts block and a grounded answer were written by hand for two domains, as templates over five slots: street number, price, opening hours, phone number and (shop only) the return window. Two extra sentences per language were written for the appended-claim variants. The benchmark then plants exactly one error per variant by substituting a slot value (30 → 25 AZN; 9:00–19:00 → 9:00–17:00; No. 12 → 14; phone ...01 02 → ...01 03; 14 → 30 days) or appending a sentence (an invented Sunday opening; an added claim such as free parking). Because the substitution is mechanical, every language receives the same seven errors and the same grounded control; the only thing that varies is the language. Eleven languages × two domains × seven answers = 154 cases, of which 132 contain a planted error and 22 are grounded. The author is a native speaker of Azerbaijani and Russian and reads the others to varying degrees; the cases are short and formulaic by design, so that reading fluency, not idiom, is what is tested. Native-speaker review of the nine other languages is invited and has not yet happened.

Scoring. A planted error is *caught* when the detector reports the answer as not grounded *and* names the planted claim: for LLM judges and LettuceDetect, the changed slot value must appear verbatim in a reported claim or span, and an appended sentence must share at least half its words with one; HHEM returns only a score, so for HHEM a flagged answer counts as caught. A grounded answer reported as not grounded is a *false alarm*. An empty or unparsable reply is *unjudged* and counts as a miss.

The judge. One chat completion at temperature 0 over any OpenAI-compatible endpoint. The system prompt carries the sources as numbered facts and asks for every factual claim in the answer that the facts do not support, and a rewrite without them, as JSON. Judges were run on Groq (openai/gpt-oss-120b and -20b, qwen/qwen3.8-27b, allam-2-7b) and Google (gemini-3.8-flash, gemini-3.5-flash, gemini-2.5-flash) on 19–20 September 2026; the baselines on a laptop CPU with default thresholds (HHEM: consistency below 0.5 is a flag; LettuceDetect: any predicted span is a flag).

3. Results

Detector	Kind	Errors caught	False alarms	Unjudged / errors	Median latency
openai/gpt-oss-120b	LLM judge (Groq)	132/132 (100 %)	0/22	0/0	0.70 s
gemini-3.8-flash	LLM judge (Google)	132/132 (100 %)	0/22	0/0	2.88 s
qwen/qwen3.8-27b	LLM judge (Groq)	131/132 (99 %)	0/22	0/0	0.36 s
openai/gpt-oss-20b	LLM judge (Groq)	130/132 (98 %)	1/22	0/0	0.45 s
gemini-3.5-flash	LLM judge (Google)	129/132 (98 %)	0/22	0/0	5.05 s
gemini-2.5-flash	LLM judge (Google)	129/132 (98 %)	0/22	0/0	5.54 s
HHEM-2.1-Open	trained classifier	104/132 (79 %)	12/22	0/0	0.05 s
allam-2-7b	LLM judge (Groq)	67/132 (51 %)	21/22	9/0	0.32 s
LettuceDetect-base (en)	trained classifier	65/132 (49 %)	11/22	0/0	0.06 s

Table 1. All 154 cases. Planted errors caught out of 132; grounded answers wrongly flagged out of 22. Latency for the judges is per network call from Baku; for the classifiers, per inference on a laptop CPU.

Detector	price	hours	address	phone	days	sunday	extra
openai/gpt-oss-120b	22/22	11/11	22/22	22/22	11/11	22/22	22/22
gemini-3.8-flash	22/22	11/11	22/22	22/22	11/11	22/22	22/22
qwen/qwen3.8-27b	22/22	11/11	21/22	22/22	11/11	22/22	22/22
openai/gpt-oss-20b	22/22	11/11	20/22	22/22	11/11	22/22	22/22
gemini-3.5-flash	22/22	8/11	22/22	22/22	11/11	22/22	22/22
gemini-2.5-flash	22/22	8/11	22/22	22/22	11/11	22/22	22/22
HHEM-2.1-Open	22/22	8/11	17/22	12/22	5/11	20/22	20/22
allam-2-7b	18/22	3/11	9/22	10/22	6/11	8/22	13/22
LettuceDetect-base (en)	9/22	8/11	11/22	5/22	0/11	15/22	17/22

Table 2. Recall by error type, all languages. Wrong price, hours, street number, phone digit and return window are slot substitutions; "sunday" and "extra" are appended sentences. Hours applies to the clinic only and days to the shop only, hence 11 each.

Detector	en	az	ru	tr	uk	kk	ar	fa	hi	id	vi
openai/gpt-oss-120b	12/12	12/12	12/12	12/12	12/12	12/12	12/12	12/12	12/12	12/12	12/12
gemini-3.8-flash	12/12	12/12	12/12	12/12	12/12	12/12	12/12	12/12	12/12	12/12	12/12
qwen/qwen3.8-27b	12/12	12/12	12/12	12/12	12/12	12/12	12/12	12/12	11/12	12/12	12/12
openai/gpt-oss-20b	12/12	12/12	12/12	12/12	12/12	11/12	12/12 ·1	12/12	12/12	12/12	11/12
HHEM-2.1-Open	5/12	10/12 ·1	12/12 ·2	7/12 ·1	9/12 ·1	7/12	12/12 ·2	12/12 ·2	11/12 ·1	12/12 ·2	7/12
LettuceDetect-base (en)	8/12	8/12	6/12 ·1	8/12 ·2	5/12 ·1	2/12 ·2	8/12 ·2	6/12 ·1	2/12 ·1	6/12 ·1	6/12

Table 3. Recall by language (caught / 12 planted) and, after the dot, false alarms out of 2 grounded answers. en English, az Azerbaijani, ru Russian, tr Turkish, uk Ukrainian, kk Kazakh, ar Arabic, fa Persian, hi Hindi, id Indonesian, vi Vietnamese.

The judges. openai/gpt-oss-120b and gemini-3.8-flash catch all 132 planted errors with no false alarms in all eleven languages; qwen/qwen3.8-27b misses one and gemini-3.5-flash three, still with no false alarms. The scripts — Latin with heavy diacritics, Cyrillic, Arabic, Perso-Arabic, Devanagari — make no visible difference to these models. The 7B allam-2-7b is the exception among judges: it over-flags grounded answers and misses errors in every language including English, which is an instruction-following limit rather than a language limit. Among the judges, harder error types separate the weaker models: the single changed phone digit and the wrong hours account for most of the misses (Table 2).

HHEM-2.1-Open. Its headline recall, 104/132, is not what it looks like. In English it catches 5 of 12 with no false alarms — a genuine but weak judgement on these short, number-heavy cases. In Russian, Arabic, Persian and Indonesian it catches 12 of 12 *and* flags both grounded answers: its consistency score for any non-English premise-hypothesis pair sits around 0.4, below the default threshold, whatever the content. The model is not detecting an unsupported claim; it is detecting that the text is not English. A detector that flags everything in a language has perfect recall and no use in it. Its false-alarm count, 12/22, is the honest number.

LettuceDetect. Recall follows script distance from its training data: 8 of 12 in English, Azerbaijani and Turkish, 6 in Russian, 2 in Kazakh and 2 in Hindi, with false alarms in eight of the eleven languages. It does name spans, so where it works it is as actionable as a judge, and at 0.06 s on a CPU it is an order of magnitude cheaper. For a Latin-script language with a tolerance for misses it remains a reasonable first filter; for Cyrillic, Arabic-script or Devanagari text it is not.

4. A measurement pitfall

An earlier run of this benchmark reported gemini-2.5-pro at 0 of 22 errors caught with 0 false alarms — a perfect-looking failure. Thinking models spend the output budget on reasoning before the first visible character; with a 1,200-token budget the reply came back empty, and the first version of the package scored an empty reply as "no unsupported claims", i.e. grounded. The corrected package returns *judged = false* for an empty or non-JSON reply, the benchmark counts it as a miss, and the budget is 8,000 tokens with one retry at the server's cap when a small model rejects it. A second, quieter version of the same mistake appeared in this report's baseline run: HHEM's score was first recorded as a "span" and then required to overlap the planted text, which no number can, giving 0 of 132; the raw scores were kept, so rescoring did not require rerunning. Both are reported because an evaluation that only publishes the tables that looked right is not one.

5. Limitations

The benchmark is formulaic by construction: short answers, numeric slots, one error each. It measures whether a detector reads the language and follows the instruction; the near-perfect judge scores mean "not a language problem" and must not be read as "solved". Harder failures — a right number in the wrong sentence, a claim that is implied but not stated, an omitted condition, a paraphrase that changes meaning — are the next set and will lower every row of Table 1. The nine non-native languages were written by one author and await native review. Default thresholds were used for the classifiers; tuning HHEM's threshold per language would raise its English recall and would not repair its behaviour on non-English text, whose scores collapse regardless of content. Only two providers of judges were run, for cost; the runner accepts any OpenAI-compatible endpoint.

6. Availability

Package (*pip install groundedness*, Python 3.9+, no dependencies), the case templates, both runners, the raw per-call outputs for every model and detector, and this report's generator are at github.com/aghasalim/groundedness under the MIT licence. The judge described here runs on every channel of SIBA's assistant (siba.az), where it reports the share of answers that passed; that deployment is the origin of the method.

Reproduce.

```
pip install groundedness pytest
GROQ_API_KEY=... python benchmark/run_v2.py
GEMINI_API_KEY=... python benchmark/run_v2.py --gemini
pip install torch transformers lettucedetect && python benchmark/run_baselines.py
```

References

- Vectara. HHEM-2.1-Open and the Hallucination Leaderboard. huggingface.co/vectara/hallucination_evaluation_model; github.com/vectara/hallucination-leaderboard.
- Kovács, Á. et al. LettuceDetect: A Hallucination Detection Framework for RAG Applications. arXiv:2502.17125, 2025.
- Ravi, S. et al. Lynx: An Open Source Hallucination Evaluation Model. arXiv:2407.08488, 2024.
- Niu, C. et al. RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models. ACL 2024.
- Manakul, P., Liusie, A., Gales, M. SelfCheckGPT. EMNLP 2023.
- Farquhar, S. et al. Detecting hallucinations in large language models using semantic entropy. Nature 630, 2024.
- Aghayev, F., Ahmadbayli, E. SIMURG: Zero-Leak Online Detection of LLM Decoding Corruption in Production Streams. HAL-X AI, 2026.